Movie Search on Spin/WASM and OCI

Comparing WebAssembly as isolation for microservices

LSC Project

24 May 2026

Abstract

This project compares the same HTTP application running under two isolation models: Spin/wasmtime as a WASI component and a native Rust server packaged as an OCI image. After an initial analysis, we abandoned comparing Spin with full Meilisearch, because official Meilisearch relies on LMDB and assumptions typical of native runtimes that could not be ported 1:1 to WASI within the project timeline. Instead, we built a custom, deterministic `movie-search` service in which Spin and OCI share the same application code, fixture, and API semantics. The service now includes opt-in Meilisearch-inspired features such as filters, facets, sorting, highlighting, suggestions, and lightweight typo tolerance while preserving the original benchmark payloads. Results show full functional parity, a shorter first search request on Spin, very fast OCI responses for a simple empty query, and a clearly larger observed Spin process RSS under load. We conclude that Spin/WASM is a reasonable candidate for isolated, short-lived HTTP microservices, but OCI containers remain a more mature and predictable operational choice, especially for storage-heavy services.

1 Goal and context

The project examined whether WebAssembly run via Spin and wasmtime can serve as a practical isolation substrate for serverless-style microservices compared with a classic OCI container. WebAssembly is a binary instruction format designed as a portable compilation target for programming languages, including outside the browser [1]. WASI extends this model with system interfaces such as files, networking, clocks, and randomness while keeping a controlled permission model [3, 2]. Spin is a framework for building WebAssembly microservices and applications, with a simple `spin build` and `spin up` workflow, HTTP triggers, and WASI-compatible languages [4]. Wasmtime acts as the runtime for WebAssembly, WASI, and the Component Model in this stack [3].

The reference point is OCI: an open standard for describing and running container images, built around runtime, image, and distribution specifications [5]. A Docker container packages an application and its dependencies into a standardized unit that is isolated from the host and portable across environments [6]. The research question is therefore: what do we get when we run the same application code once as a WASI component and once as a native process in a container?

2 Pivot away from Meilisearch

The first direction assumed comparing Meilisearch in a container with a Spin-hosted variant. A feasibility analysis showed that this would be technically unfair. Official Meilisearch stores data in `data.ms` and uses LMDB as its storage engine; LMDB relies on memory-mapped files [7]. Meilisearch backups distinguish snapshots, i.e. a copy of the `data.ms` database, from dumps portable across Meilisearch versions [8]. That remains Meilisearch's data model, not a portable index for a custom WASI component.

In addition, local attempts to compile upstream Meilisearch layers to `wasm32-wasip2` stopped on dependencies such as `ring`, Tokio, and expected storage blockers related to LMDB and mmap.

The project therefore shifted from "port Meilisearch" to "compare the same custom microservice under two runtime isolations." The benchmark thus measures deployment and isolation differences, not search-engine semantics.

3 Application and architecture

The final application is `movie-search`: a compact HTTP service written in Rust. Both runtimes use the same `movie-search-core` library, the same `fixtures/movies.json` file, and the same ranking rules. The fixture contains 44 471 unique movies after deduplication by `id`. Search is deterministic: tokenization on non-alphanumeric characters, case-insensitive substring matching, sorting by the number of matched tokens, field weights `title > genre > overview`, and finally ascending `id`. Empty queries still return all documents by ascending `id`; this preserves the original benchmark semantics.

To make the demo closer to a real search product without compromising fairness, we added an optional Meilisearch-inspired feature layer inside the shared core. `POST /search` accepts filters over genre and year, deterministic sorting by year, title, or id, facet distribution for genre, year min/max statistics, escaped highlights using `<mark>`, and opt-in typo tolerance based on Levenshtein distance. Both adapters also expose `POST /suggest`, which ranks title prefix matches before title substring matches. These features are not a reimplementation of Meilisearch, but they make the practical API more impressive while keeping Spin and OCI on exactly the same application code path.

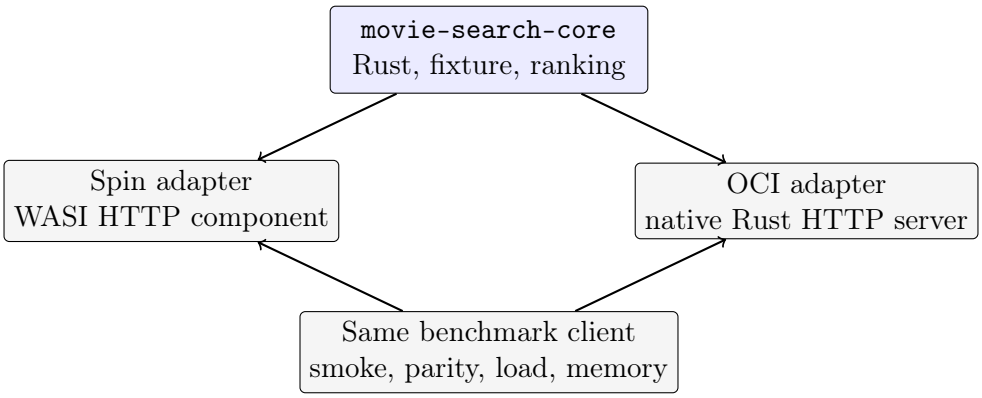

Figure 1: Final architecture: one application core, two runtime adapters.

The public API is shared by both variants: `GET /health`, `GET /version`, `GET /stats`, `GET /movies?offset=&limit=`, `POST /search`, and `POST /suggest`. Spin runs locally on port 8080, and the OCI variant on port 8081.

4 Methodology

Before performance measurement, we run unit tests, build the Spin component, build the OCI image, and run smoke/parity checks. The script `benchmarks/compare_results.sh` compares engine, document count, hit count, and `hit.id` lists for queries `space`, `toy story`, `dark knight`, `romance`, and the empty query. It also compares one enhanced payload with filters, facets, highlighting, typo tolerance, and suggestions.

The full benchmark was run with `benchmarks/run_all.sh`. It includes 20 cold-start repetitions, load tests on `POST /search` for queries `space`, empty, and `enhanced` at concurrency 10, 50, 100, and 200, and memory samples in idle and under-load states. The historical tables in this report show the legacy `space` and empty scenarios; the current harness now records the enhanced scenario as a separate CSV label so it can be analyzed without overwriting earlier results. Environment: Apple M4, 10 cores, 24 GiB RAM, macOS 15.6, Spin 3.6.3, Docker 28.5.1, Rust 1.95.0. The Docker image was built before the cold-start measurement; build time is not part of the result.

An important limitation concerns memory. OCI is measured via `docker stats` and reflects the container's perspective. Spin is measured as host process-tree RSS. These numbers are useful for comparing observed overhead but are not an identical memory accounting layer.

5 Results

All functional and load tests completed without response validation errors. The parity check confirmed identical functional results for both runtimes.

Table 1: Cold start: 20 repetitions per system.

System	Metric	Median [ms]	p95 [ms]	p99 [ms]	Errors
OCI	`/health`	20.982	28.420	36.325	0
OCI	`/health + /search`	307.755	376.983	446.288	0
Spin	`/health`	141.357	144.220	152.354	0
Spin	`/health + /search`	205.649	217.504	218.947	0

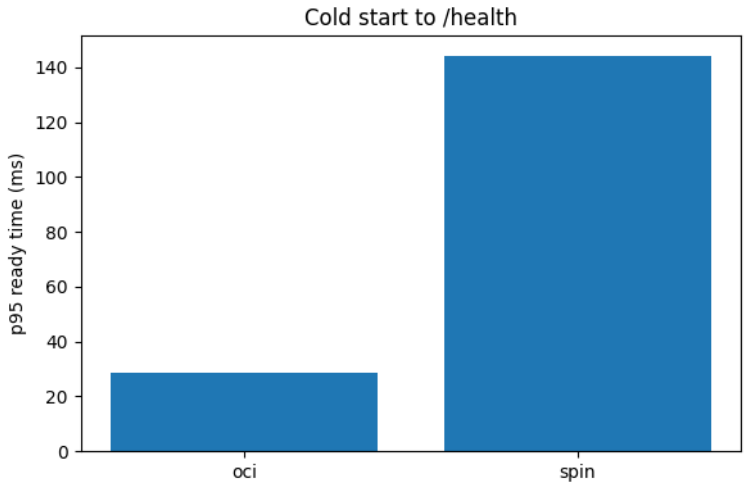

Figure 2: p95 cold start until the first successful `/health`.

In this environment, the container reached `/health` readiness faster, while Spin completed the cold start plus first search sequence faster. This should be interpreted as a property of the specific implementation and measurement approach: `/health` is a cheap readiness endpoint, whereas `/search` exercises the application's real work path.

Table 2: Load test: request rate and p95 latency.

System	Query	c=10		c=50		c=100		c=200	
		req/s	p95 ms	req/s	p95 ms	req/s	p95 ms	req/s	p95 ms
OCI	empty	3314.9	5.4	2931.1	26.8	2934.6	52.3	2899.0	95.7
Spin	empty	132.8	100.0	129.1	643.5	123.0	1516.1	123.3	3636.0
OCI	space	54.0	210.8	46.2	2164.9	58.0	1899.6	61.5	3747.5
Spin	space	77.2	168.3	70.3	1264.6	74.5	2491.2	78.3	6073.3

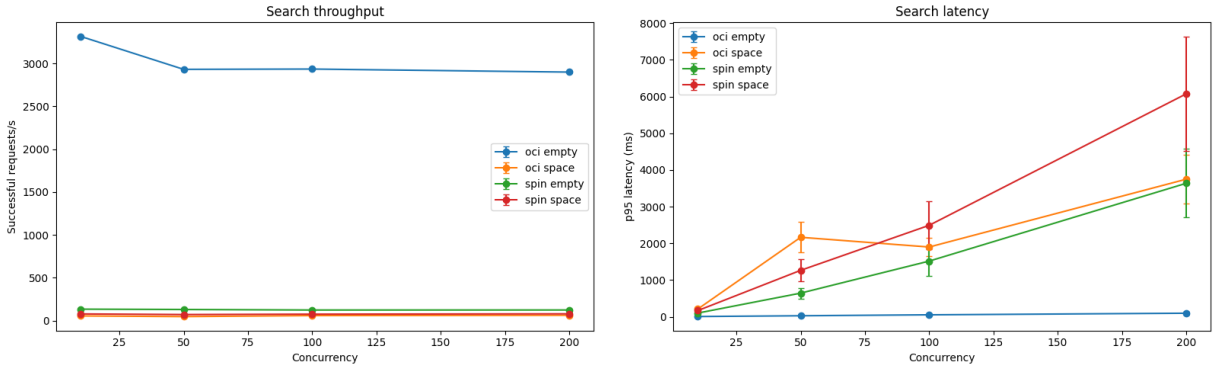

Figure 3: Throughput and p95 latency for `POST /search`.

For the empty query, the native OCI server was decisively faster. This is a low application-cost path: it returns the first documents deterministically without expensive text filtering. For the `space` query, the picture is more nuanced. Spin achieved higher throughput but worse latency tails at very high concurrency. This suggests the benchmark measures not only isolation but also the interaction of simple linear scanning over 44 471 records with each runtime's HTTP model and scheduling.

Table 3: Memory: maximum of observed samples.

System	Phase	Source	Max [MiB]
OCI	idle	`docker stats`	29.8
OCI	load	`docker stats`	31.9
Spin	idle	host process RSS	104.8
Spin	load	host process RSS	493.5

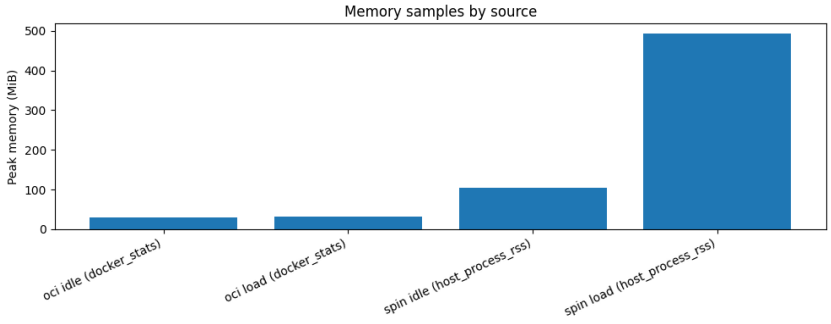

Figure 4: Peak memory samples for idle and load.

Memory is the strongest argument against the simple claim that the WASM variant is always lighter. In this project, the observed Spin process together with wasmtime had higher RSS than the OCI container. At the same time, this is not a full production cost model, because Docker reports memory from the container perspective while Spin is measured from the host process perspective. The final report therefore draws a cautious conclusion: Spin offers an interesting capability-oriented isolation model, but it requires deliberate measurement and does not automatically guarantee lower overhead.

6 Demo

The presentation demo does not run the full benchmark live. Measurements are collected beforehand; during the presentation we show a reproducible path:

1. start Spin on `127.0.0.1:8080` and OCI on `127.0.0.1:8081`;
2. check `/health`, `/version`, and `/stats`;
3. run one `space` search;
4. run an enhanced search with genre/year filters, typo tolerance, highlights, and facets;
5. call `/suggest` for title autocomplete;
6. run `benchmarks/compare_results.sh`;
7. show the final charts and CSV files.

If the live environment fails, the fallback plan is to show saved artifacts: `results/raw`, `results/processed`, and `results/plots`. The full scenario is in `demo/demo-script.md`.

7 Reproducibility and limitations

The repository is set up so results can be reproduced in a single run:

```
benchmarks/run_all.sh
```

The script builds both variants, runs correctness tests, performs smoke/parity checks, collects three kinds of raw CSV data, and generates processed files: `results/processed/cold_start_summary.csv`, `results/processed/load_summary.csv`, and `results/processed/memory_summary.csv`. Charts in this report come from `results/plots`. Raw data from the final run are kept in `results/raw`; earlier pilot data are not mixed with the final results.

There are four main interpretive limitations. First, the application does not use an inverted index: even the enhanced search path is still deterministic fixture scanning, so it does not compare production search-engine quality. Second, typo tolerance and highlighting are lightweight feature demonstrations, not a replacement for Meilisearch's ranking engine. Third, cold-start measurement depends on the local host, system cache, and the fact that the OCI image was built beforehand. Fourth, Spin and OCI memory are measured from different tooling perspectives. Conclusions should therefore be treated as the outcome of a controlled local experiment, not a universal characterization of all WASM and container workloads.

8 Conclusions

The most important project outcome is methodological: the comparison is fair because both runtimes execute the same application code and return identical results. Spin/WASM fits small, isolated HTTP components, plugins, edge functions, and serverless workloads where portability, sandboxing, and a simple capability-oriented model matter. OCI containers remain more mature operationally: they have widespread tooling, stable resource accounting, a familiar deployment model, and very good performance for simple native paths.

In our benchmark, OCI had faster `/health` readiness, Spin had a faster first search request after startup, and load results depended strongly on the query. The empty query clearly favored OCI, while the `space` query showed higher Spin throughput with worse latency tails at high concurrency. The enhanced feature path gives a more realistic demo surface and is now part of the benchmark harness, but it remains fair because the implementation lives in the shared core. The Meilisearch attempt remains a valuable appendix: a storage-heavy system built on LMDB and mmap is not a good candidate for a simple WASI port without substantial architectural changes.

References

[1] WebAssembly, *Official website*. https://webassembly.org/.

[2] WASI.dev, *Interfaces*. https://wasi.dev/interfaces.

[3] Bytecode Alliance, *Wasmtime Introduction*. https://docs.wasmtime.dev/introduction.html.

[4] Fermyon, *Develop serverless WebAssembly apps with Spin*. https://www.fermyon.com/spin.

[5] Open Container Initiative, *Open Container Initiative*. https://opencontainers.org/.

[6] Docker, *What is a Container?*. https://www.docker.com/resources/what-container/.

[7] Meilisearch, *Storage*. https://www.meilisearch.com/docs/resources/internals/storage.

[8] Meilisearch, *Backing up Meilisearch data*. https://www.meilisearch.com/docs/resources/self_hosting/data_backup/overview.